Содержание

Power Systems Agent Benchmark: исполняемый бенчмарк для оценки инструментальных ИИ-агентов в задачах электроэнергетики **1**
Аннотация . . . 1
1. Введение . . . 2
2. Связанные работы . . . 3
3. Требования к бенчмарку для электроэнергетических агентов . . . 4
4. Архитектура . . . 4
4.1 Контракт артефактов . . . 4
4.2 Агент-агностичный запуск и устойчивость harness . . . 4
4.3 Лестница зрелости оценщиков . . . 5
4.4 Скрытый срез как генератор, а не как файлы . . . 5
5. Набор задач . . . 5
5.1 Доменная выверка набора . . . 6
6. Проверяющие программы и метрики . . . 6
7. Воспроизводимость . . . 6
8. Эксперименты . . . 7
8.1 Канонический раунд (public vs hidden) . . . 7
8.2 Типология провалов: способности против спецификации . . . 7
8.3 Целостность оценки: агент, добравшийся до оракула . . . 8
9. Ограничения . . . 8
10. Заключение . . . 9
Приложение A. Каталог задач . . . 9
Приложение B. Артефакты экспериментов . . . 9

Power Systems Agent Benchmark: исполняемый бенчмарк для оценки инструментальных ИИ-агентов в задачах электроэнергетики

Черновик препринта v2 (2026-06). Русская версия. Дословная англоязычная пара: `paper_en_v2.md`*. Аннотированная библиография:* `bibliography.md`*; BibTeX:* `references.bib`*.*

Аннотация

В работе представлен Power Systems Agent Benchmark — прототип исполняемого бенчмарка для оценки инструментальных ИИ-агентов в задачах электроэнергетики. В отличие от текстовых question-answering наборов и оценки отчётов через LLM-as-a-Judge, наш протокол требует от агента сформировать структурированное решение (`submission.json`), которое затем проверяется детерминированной программой: она пересчитывает инженерные величины, проверяет физические и эксплуатационные ограничения и возвращает флаг допустимости, нормированный балл и машиночитаемый список нарушений. Публичный срез содержит 41 семейство задач, охватывающих расчёт режимов и анализ сети, короткие замыкания и релейную защиту, устойчивость и соответствие grid code, DER/EV и накопители, диспетчеризацию микросетей, надёжность и восстановление, качество электроэнергии и стандарты, кибербезопасность измерений, прогнозирование. Скрытый срез не распространяется: вместо него публикуется детерминированный генератор, порождающий скрытые кейсы из приватного seed с автоматической проверкой решаемости, так что механизм аудируем, а конкретные числа остаются приватными. На матрице из CLI-агентов мы показываем, что (1) бенчмарк различает агентов согласованно на публичном и скрытом срезах; (2) цикл «прогон — анализ идентичных провалов — уточнение постановок — повторный прогон» отделяет дефекты спецификации от настоящих различий в способностях и вскрыл десять дефектов постановок и — что

особенно показательно — латентный баг физики в проверяющем коде задачи, невидимый для проверки «baseline = evaluator» и обнаружимый только потому, что независимые агенты применили корректную доменную физику. На исправленных постановках публичный и скрытый срезы дают согласованный рейтинг и сопоставимые баллы; ценность скрытой оценки — проспективная защита от контаминации, а не разовый разрыв сложности. Текущие проверки — компактные детерминированные суррогаты; переход к simulator-backed оценщикам (pandapower, OpenDSS) описан как дорожная карта, не меняющая контракта задач.

1. Введение

Большие языковые модели всё чаще применяются как агенты к инженерным задачам: они читают данные, пишут вспомогательный код, вызывают инструменты и формируют решения. Стандартные способы оценки таких агентов — текстовые вопросы с эталонным ответом либо оценка свободного отчёта другой LLM (LLM-as-a-Judge) — плохо подходят для электроэнергетики. Здесь решение может выглядеть убедительно и при этом нарушать токовые и напряженческие ограничения, селективность защиты, требования grid code или балансовые уравнения. Ошибка агента проявляется не как неправильная формулировка, а как перегрузка линии, неселективная уставка, недопустимое напряжение или неверная локализация повреждения.

Поэтому основной результат оценки должен определяться не общим впечатлением от текста, а исполняемой проверкой предложенного решения. Мы формулируем и реализуем такой протокол: каждая задача публикует структурированные входные данные (`task.json`), агент обязан вернуть машиночитаемое решение (`submission.json`), а независимый оценщик пересчитывает целевые величины, проверяет жёсткие ограничения и возвращает `feasible`, нормированный `score` и список `violations`. Свободный текст агента `rationale` сохраняется как диагностика, но не влияет на счёт.

Вклады:

1. Исполняемый контракт оценки `task.json → агент → submission.json → evaluator → evaluation_result.json` с JSON-схемами для каждого артефакта, чётким разделением жёстких ограничений (`feasible`) и качества (`score`) и агент-агностичным протоколом запуска любого CLI-агента через переменные окружения.
2. Сбалансированный публичный набор из 41 исполняемого семейства задач по восьми доменным группам, каждое с прослеживаемым происхождением (source cards, ссылки на стандарты и литературу) и двуязычным (en/ru) представлением.
3. Архитектурное решение проблемы утечки скрытого набора: вместо файлов скрытых кейсов публикуется детерминированный per-family генератор; кейсы порождаются приватным seed на хосте оценки, и каждый розыгрыш принимается, только если baseline семейства его решает и evaluator подтверждает допустимость. Механизм публично аудируем; конкретные числа — нет.
4. Методология контроля качества постановок: по матрице агентов идентичный провал всех участников трактуется как симптом дефекта постановки или оценщика, а не слабости агента. Критерий вскрыл десять дефектов постановок, которые мы исправили и закрыли повторным прогоном.
5. Сильнейший случай пункта (4): латентный баг физики в evaluator-е локализации повреждения (off-by-one в модели ФПИ), который проверка самосогласованности увидеть не могла — baseline унаследовал ту же ошибку. Вскрыли его три независимых агента, применивших корректную физику, — результат, который мы считаем наиболее ясной демонстрацией ценности метода.

Помимо набора, работа вносит переиспользуемый harness: модель изоляции, не дающую агентам добраться до оракула, устойчивость к мусорным сабмишенам, переживание крашей оценщиков и узкую политику ретраев только при транзиентных инфраструктурных сбоях, которая отдельно сообщает raw и retry-normalized метрики, так что надёжность провайдера никогда не выдаётся за способность агента (раздел

4).

2. Связанные работы

Бенчмарки агентов. Общеназначенческие наборы — AgentBench [liu2024agentbench], GAIA [mialon2023gaia], WebArena [zhou2024webarena] — оценивают веб-навигацию, код и использование инструментов, но не инженерные домены. SWE-bench [jimenez2024swebench] и Codex/HumanEval [chen2021codex] установили жанр исполняемой оценки (решение проверяется тестами); τ-bench [yao2024taubench] перенёс принцип «сравнивай конечное состояние, а не текст» на диалоговых агентов и ввёл pass^k как меру устойчивости. Наш контракт распространяет исполняемую оценку с программного кода на инженерные расчёты с физическими ограничениями.

LLM-as-a-Judge. Судейство LLM [zheng2023judging] страдает документированными смещениями — позиционным, по длине, самопредпочтением [wang2023fairevaluators, gu2024judgesurvey, li2024generationjudgment]; для задач, где ответ может выглядеть убедительно и нарушать ограничения, судья непригоден как основной счёт. Мы используем детерминированный пересчёт, а rationale храним только как диагностику.

LLM-агенты в энергетике. Обзоры [llmpowersurvey2025, agenticpowersurvey2025] фиксируют быстрый рост применений при отсутствии стандартизированной оценки. Ближайшая по концепции работа — PowerAgentBench [poweragentbench], бенчмарк-компонент экосистемы PowerAgent (Harvard SEAS) [zhang2025poweragent]: три уровня задач (N-1 аудит на IEEE 39-bus, агентный N-2 поиск, ревизия качества динамической модели с PSS/E) с public/hidden split и скрытым evaluator-ом, пересчитывающим физическую корректность; отдельная статья по бенчмарку на момент написания не вышла. Наш набор отличается охватом (41 семейство по восьми доменным группам против трёх уровней), генерацией скрытых кейсов из приватного seed вместо статического hidden-набора, агент-агностичным CLI-протоколом и формализованным циклом контроля качества постановок. Другие соседи: ElecBench [zhou2025elecbench] — диспетчерские сценарии, но с текстово-судейской оценкой; PFBench [pfbench2025] — программная проверка, но только power flow. Появляющиеся производственные агенты — GridMind [gridmind2025], X-GridAgent [xgridagent2025], Grid-Mind CIA [gridmindcia2026], PFAgent [pfagent2026] — и есть целевые «испытуемые» таких бенчмарков.

RL-среды управления сетью. Grid2Op/L2RPN [marot2020l2rpn, marot2021retrospective] — зрелая среда последовательного топологического управления для RL; мы решаем другую задачу — агент-агностичный контракт однократных инженерных решений по широкому спектру подзадач отрасли.

Симуляторы. pandapower [thurner2018pandapower], MATPOWER [zimmerman2011matpower], OpenDSS [dugan2011opendss], PowerModels/PGLib [coffrin2018powermodels, babaeinejadsarookolaee2019pglib], ANDES [cui2021andes], PyPSA [brown2018pypsa] покрывают расчётные ядра, но не задают контракта оценки агентов; в нашей дорожной карте они — substrate проверяющих программ, подключаемый без изменения формата задач.

Целостность бенчмарков. Контаминация обучающих данных систематически завышает результаты [contaminationsurvey2025, statictodynamic2025]; ответ — динамические и обновляемые наборы (Dynabench [kiela2021dynabench], LiveBench [white2024livebench], LiveCodeBench [jain2024livecodebench]). Наш скрытый генератор — prevention-механизм того же класса: мы публикуем способ порождения кейсов, а не кейсы. Критерии честной агентной оценки из [kapoor2024agentsmatter] (стоимость, воспроизводимость, разделение классов провалов) реализованы прямо в harness; ограниченность best-of-N как метрики количественно показана в [brown2024monkeys].

3. Требования к бенчмарку для электроэнергетических агентов

1. Исполнимость: решение проверяется кодом, а не сравнением строк.
2. Физическая проверяемость: evaluator пересчитывает величины и проверяет ограничения, а не сверяет с сохранённым текстом.
3. Разделение public/hidden: публичные кейсы поддерживают разработку, скрытые — честное сравнение, а утечка скрытых ответов исключена архитектурно.
4. Агент-агностичность: запуск не зависит от конкретной LLM, CLI или фреймворка; контракт — «прочитай файлы, запиши файл».
5. Прослеживаемость: у каждой задачи есть source provenance с уровнем доверия.
6. Машиночитаемый результат: `feasible`, `score $\in$ [0,1]`, `violations`, метрики.
7. Рост сложности без смены контракта: переход от формульных суррогатов к simulator-backed проверкам меняет внутренность evaluator-а, а не формат task/submission.
8. (Добавлено по результатам экспериментов) Контроль качества постановок: бенчмарк должен включать процедуру отделения дефектов спецификации от настоящих различий в способностях (раздел 8.2).

4. Архитектура

4.1 Контракт артефактов

`task.json → агент (любой CLI) → submission.json → evaluator → evaluation-result.json`

- `schemas/task.schema.json` — обязательные поля: `task_id`, `family`, `substrate`, `objective`, `public_inputs`, `submission_schema`, `hard_constraints`, `metrics`, `source_evidence`, `source_provenance`, плюс двуязычный блок.
- `schemas/submission.schema.json` — структурированный ответ: `actions` и/или `answer`, опциональный `rationale`.
- `schemas/evaluation-result.schema.json` — `task_id`, `feasible` (bool), `score $\in$ [0,1]`, `violations` (список), метрики и evidence.

Валидатор артефактов дополнительно гарантирует согласованность зеркальных копий данных (`public_inputs` и верхний уровень `task.json`) — дефект этого рода был найден и исправлен в ходе работы.

4.2 Агент-агностичный запуск и устойчивость harness

`prepare_agent_tasks.py` собирает самодостаточный пакет на задачу (`task.json` с редактированием + `instructions.md`); sample submissions и любые поля-оракулы вырезаются. `run_agent_command.py` запускает на каждый пакет произвольную shell-команду, передавая переменные окружения (`BENCH_PACKET_DIR`, `BENCH_TASK_PATH`, `BENCH_SUBMISSION_PATH`, …) с таймаутом на кейс. Три свойства делают harness заслуживающим доверия для сравнения агентов:

- Изоляция. Каждый кейс выполняется в песочнице-копии пакета вне репозитория, с песочницей в качестве рабочей директории и без выставления корня репозитория или его `PYTHONPATH`. Это закрывает путь к оракулу, наблюдавшийся на практике (раздел 8.3): агент нашёл и запустил evaluator и самопроверился против утёкшего оракула, пока пакеты лежали внутри дерева репозитория.
- Устойчивость к мусорным сабмишенам. Структурно-валидный, но мусорный сабмишен (null, строки вместо чисел, неверные формы) всегда возвращает `feasible=false` с пофилдовыми нарушениями, а не падает — свойство, проверенное зондом по всем 41 семействам. Полевой репорт (агент вернул null во всех числовых полях) ранее оборвал прогон на 27/41.

- Разделение классов провалов и ретраи. Узкая политика ретраев (`--max-attempts`) повторяет только транзиентные инфраструктурные сбои (сигнатуры провайдера/транспорта, ранние тихие обрывы, сигнатуры quota/spend-limit); quality-провалы, нарушения контракта и таймауты по бюджету кейса не ретраятся, потому что это молча превратило бы бенчмарк в best-of-N. Каждая строка результата хранит `attempt_count`, `failure_class_initial`, `selected_attempt` и `raw_first_attempt`, а summary сообщает и raw, и retry-normalized метрики плюс счётчик `infra_unresolved`.

4.3 Лестница зрелости оценщиков

формульный/графовый суррогат → детерминированная инженерная модель → simulator-backed скрытый оценщик → стохастический/масштабный набор сценариев.

Текущие публичные оценщики на первых двух ступенях; контракт позволяет заменять внутренность проверки (например, DC-поток на pandapower AC-решение) без изменения формата задач.

4.4 Скрытый срез как генератор, а не как файлы

Каталог `tasks/*/hidden/` исключён из системы контроля версий. Вместо него репозиторий содержит `generate_hidden_cases.py`: реестр детерминированных per-family возмущений. Кейс порождается из публичного `case_001` серией физически осмысленных перевыборок (параметры сети, погодные ряды, профили нагрузки, место повреждения с согласованными показаниями индикаторов, …), заданных приватным ключом; розыгрыш принимается, только если baseline семейства его решает и evaluator подтверждает допустимость, иначе выполняется детерминированная перерисовка. Оператор leaderboard хранит seed в секрете; любой обладатель репозитория может проверить, как порождаются скрытые кейсы, не зная их конкретных чисел.

5. Набор задач

41 публичное семейство сгруппировано в восемь категорий:

1. Анализ сети и ограничения режима — `ybus_construction`, `pf_voltage_repair`, `contingency_screening`, `corrective_transmission_switching`, `dynamic_thermal_rating`.
2. Короткие замыкания и защита — `three_phase_short_circuit` (пара максимального/минимального тока по МЭК 60909), `earth_fault_calculation`, `breaker_relay_short_circuit`, `distance_protection_settings` `overcurrent_protection`, `cable_ampacity_sizing`.
3. Устойчивость, grid code и IBR — `critical_clearing_time` (критерий равных площадей), `transient_stability_prediction`, `frt_compliance`, `ibr_short_circuit_frt`, `min_synchronous_share`.
4. DER, PV, EV и накопители — `pv_volt_var`, `ev_v2g_outage_schedule`, `ev_v2g_voltage_support`, `bess_ancillary_response`, `commercial_pv_lcoe_uncertainty`.
5. Микросети и диспетчеризация — `microgrid_economic_dispatch`, `rolling_microgrid_dispatch`, `islanded_microgrid_pq_dispatch`, `dispatch_uncertainty`, `hydro_thermal_storage_uc`, `n1_generation_commitm`
6. Надёжность и восстановление — `flisr_restoration`, `fault_section_localization`, `fci_placement`, `fci_saidi_caidi`, `operator_breaker_load_actions`.
7. Качество электроэнергии, стандарты, активы, кибербезопасность — `en50160_voltage_compliance`, `power_quality_event_classification`, `harmonic_ieee519_compliance`, `transformer_thermal_loading` (МЭК 60076-7), `fdi_state_estimation` (WLS + тест наибольшей нормированной невязки), `protected_meter_placement`, `mas_frequency_handover`.
8. Прогнозирование и неопределённость — `wind_power_forecast`, `wind_prediction_interval`.

Каждая задача снабжена source card и source provenance с уровнем доверия; полный каталог — в приложении (`bilingual_task_inventory.md`).

5.1 Доменная выверка набора

Набор прошёл доменную ревизию, устранившую четыре класса фактических дефектов (исправления зафиксированы в истории репозитория):

- Физические ошибки: формула RoCoF помещала `f0` в знаменатель вместо числителя (расхождение `f0^2`), из-за чего ограничение по инерции никогда не было активно; исправлено по уравнению движения с перебалансировкой кейса.
- Концептуальные подмены: задача «обнаружения FDI» несла метки атак во входах и сводилась к сравнению с порогом; переработана в полноценную WLS-оценку состояния с итеративным тестом наибольшей нормированной невязки, где эффект размазывания делает наивный взвешенный тест неверным, а скрытные FDI явно вынесены за скоуп.
- Вырожденные задачи: расчёт критического времени отключения был чтением готовых флагов устойчивости из таблицы; заменён аналитическим расчётом по критерию равных площадей.
- Завышенные ссылки на стандарты: «расчёт по МЭК 60909» без коэффициента напряжения `c` и с сопротивлением дуги в максимальном токе занижал ток на ~10% неконсервативно; переработан в каноническую пару Ik_max (c_max, металлическое КЗ) / Ik_min (c_min, с сопротивлением).

Кроме того, три непрофильных семейства (теплоснабжение зданий ×2, дисперсионное разложение Соболя) заменены на ранее непокрытые профильные домены: тепловой режим трансформатора (МЭК 60076-7), выбор сечения кабеля по трём проверкам и проверка гармоник по IEEE 519.

6. Проверяющие программы и метрики

Поля результата: `feasible` (жёсткие ограничения выполнены), `score $\in$ [0,1]`, `violations` (конкретные нарушения словами) и task-specific evidence. Подходы к score по типам задач:

- точный расчёт: 1/0 или допуски на числовые поля;
- управление/оптимизация: допустимость по ограничениям + качество по целевой функции. Для задач с неединственным оптимумом (диспетчеризация микросетей) evaluator пересимулирует поданное решение и оценивает разрыв стоимости с оптимумом: равные по стоимости альтернативные оптимумы получают полный балл, допустимые субоптимальные решения — частичный;
- классификация/локализация: точные множества с перечислением промахов;
- стандарты и grid code: pass/fail + диагностические поля.

Метрики агрегируются двумя способами: feasible count (доля задач, удовлетворяющих жёстким ограничениям) и perfect count (доля со `score = 1.0`); как показано в разделе 8, они могут расходиться в рейтинге, и мы рекомендуем сообщать обе вместе с `mean_score`.

7. Воспроизводимость

`scripts/check.sh` прогоняет набор юнит-тестов (оценщики, baseline-ы, harness, генератор скрытых кейсов), валидацию артефактов, поштучную оценку всех 41 публичных sample submissions и прогон публичных baseline-ов. Baseline-ы построены на тех же вычислительных ядрах, что и оценщики, и дают 41/41 feasible со средним баллом 0.997 — это проверка самосогласованности контракта, а не свидетельство сложности задач, и это различие проговаривается в каждом отчёте. Как показывает раздел 8.2, такая самосогласованность не способна валидировать корректность оценщика: baseline, выведенный из багованного оценщика, с ним согласуется. Непрерывная интеграция прогоняет полную проверку на каждый коммит.

8. Эксперименты

CLI-агенты, идентичные промпты, таймаут 600 с на кейс и политика ретраев `--max-attempts 2`; агенты запускались без какой-либо адаптации под бенчмарк:

- Codex CLI 0.139.0, модель gpt-5.5 (reasoning effort medium);
- Cursor agent CLI, модель composer-2.5;
- OpenCode 1.15.13, модель deepseek-v4-flash.

8.1 Канонический раунд (public vs hidden)

Эталонный раунд (`runs/canonical_round1/`), проведённый после всех исправлений. Public — на зафиксированном `case_001` для семейства; hidden — на одном сгенерированном `case_401` на семейство от свежего приватного seed. Ноль infra-провалов; raw и retry-normalized метрики совпали.

агент (модель)	public feasible	public perfect	public mean	hidden feasible	hidden perfect	hidden mean
codex (gpt-5.5 medium)	40/41	36/41	0.971	40/41	37/41	0.969
cursor (composer-2.5)	39/41	35/41	0.947	40/41	36/41	0.969
opencode (deepseek-v4-flash)	37/41	35/41	0.900	38/41	35/41	0.920

Два наблюдения. Бенчмарк различает агентов: codex ≈ cursor (~0.97 mean) заметно выше opencode (~0.90–0.92), устойчиво на обоих срезах. На исправленных постановках public и hidden сопоставимы: у каждого агента hidden-балл в пределах шума от public. Драматичное расхождение рейтинга, наблюдавшееся в раннем dev-раунде — где лидер public будто бы рушился сильнее всех на hidden — целиком объясняется дефектами постановок, непропорционально бившими одного агента; после их исправления и устранения латентного бага оценщика (раздел 8.2) срезы согласуются. Поэтому мы не заявляем разовый разрыв сложности public/hidden. Роль скрытой оценки — проспективная: защита от контаминации по мере старения публичного набора и обучения моделей на нём. Как косвенное подтверждение: эти свежие модели не показывают на опубликованном public-наборе балл выше, чем на свежесгенерированных hidden-кейсах — признак запоминания для столь компактных кейсов отсутствует.

8.2 Типология провалов: способности против спецификации

Ранняя матрица показала две различные группы провалов. Группа A — агенты расходятся: нарушения лимитов напряжения в Volt-VAR, неверный FRT-вердикт, перепутанный знак разряда EV, разные ошибки в risk-скоринге устойчивости; это сигнал лидерборда. Группа B — все агенты падают идентично; это симптом дефекта спецификации. Мы выявили десять дефектов за два прохода: описание поля, противоречащее его имени (доля против процентов — два агента вернули 0.51 вместо 51.0 при численно верном расчёте); имена вложенных ключей, не совпадающие с проверкой; отвержение семантически эквивалентной нулевой записи; требование точного совпадения с одним из нескольких равных по стоимости оптимумов; невысказанные конвенции кривой мощности ветра, квантилей и членов теплового баланса провода; зашитая P/Q-политика диспетчеризации; и неугадываемая формула risk-скоринга. Критерий «идентичный

провал всех участников ⇒ подозрение на дефект спецификации или оценщика» предлагается как стандартная процедура контроля качества исполняемых бенчмарков. После каждого исправления затронутые семейства перепрогонялись (или существующие сабмишены переоценивались), и те же агенты восстанавливались, подтверждая диагноз.

Сильнейший случай возник в каноническом раунде и касается самого оценщика. На скрытом розыгрыше `fault_section_localization` все три агента локализовали повреждение в секцию s24 — согласованно и с индикаторами тока КЗ, и с жалобами на отключение, — а evaluator требовал другую секцию, противоречащую собственным показаниям индикаторов кейса. Причина — off-by-one в модели ФПИ оценщика: он помещал нижний узел повреждённой секции на путь тока КЗ, из-за чего индикатор в этом узле ложно регистрировал повреждение, тогда как индикатор видит ток КЗ только если повреждение строго ниже него. Принципиально: баг был невидим для собственной проверки самосогласованности бенчмарка: baseline выводится из той же функции, что и evaluator, поэтому baseline и evaluator согласовались друг с другом, и публичный sample проходил. Вскрыть его могли только независимые агенты с корректной физикой. Мы считаем это — исполняемый бенчмарк, в оценщике которого был латентный доменный дефект, пойманный матрицей агентов, — наиболее ясной демонстрацией ценности методологии и предостережением: построение baseline из оценщика не способно валидировать корректность самого оценщика.

8.3 Целостность оценки: агент, добравшийся до оракула

Аудит логов скрытого прогона выявил, что один агент (Cursor) на задаче устойчивости нашёл и запустил проверяющую программу и самопроверился против оракула: его лог сообщает, что сабмишен «оценивается как feasible со score 1.0». Это стало возможным потому, что пакеты лежали внутри рабочего дерева репозитория, а harness запускал агентов с репозиторием в качестве рабочей директории; запрет существовал только в инструкциях пакета. Урок — изоляция на уровне инструкций недостаточна для агентов с автономным доступом к файловой системе — привёл к переработке песочницы в разделе 4.2. Находка включена как самостоятельный методологический результат.

9. Ограничения

1. Публичные кейсы компактны (сети 3–5 узлов, короткие ряды): они проверяют владение методом, а не масштаб промышленных данных.
2. Оценщики — детерминированные суррогаты; промышленную точность АС-расчётов нельзя заявлять до подключения simulator-backed оценщиков.
3. По одному кейсу на семейство в каждом срезе: ранжирование по 41 точке статистически хрупко; генератор делает наращивание выборки дешёвым.
4. Baseline 41/41 — проверка самосогласованности контракта, а не нижняя граница сложности, и не способна валидировать корректность оценщика.
5. Seed любого раскрытого скрытого раунда «сжигается» прогоном; цитируемый leaderboard-раунд требует свежего seed на изолированном хосте.
6. Песочница реализована переносом, а не контейнеризацией: она устраняет утечку через относительные пути и окружение, но не противостоит намеренному поиску оракула по абсолютному пути; соревновательные условия требуют контейнеров на уровне хоста.
7. Одна конфигурация на агента; без повторов дисперсия по сэмплингу LLM не оценена.
8. Попытка оценить в рамках того же семейства собственную модель мейнтейнера на аккаунте с общим бюджетом инфраструктурно несостоятельна (исчерпывает бюджет сессии) и исключена; чистая самооценка требует выделенного API-ключа и явной оговорки о конфликте интересов.

10. Заключение

Power Systems Agent Benchmark демонстрирует практический путь от текстовых задач к исполняемой оценке ИИ-агентов в электроэнергетике. Предложен и реализован воспроизводимый benchmark contract, который переводит электроэнергетические задачи из формата текстовых ответов в формат исполняемой проверки; показано, что он различает современных агентов согласованно на публичном и скрытом срезах. Центральный методологический вклад — цикл контроля качества по матрице агентов: трактовка идентичного провала всех агентов как сигнала дефекта вскрыла десять дефектов постановок и латентный баг физики в оценщике задачи, который проверка самосогласованности поймать не могла, — переносимая процедура для любого исполняемого бенчмарка и предостережение, что baseline, выведенный из оценщика, не валидирует сам оценщик. Главная ценность — не в конкретных компактных кейсах, а в контракте, таксономии, проверяющих программах, seed-генераторе скрытых кейсов (чья роль — проспективная защита от контаминации), изолированном harness и воспроизводимом цикле контроля качества; дорожная карта — постепенное наполнение проверок simulator-backed реализациями без изменения внешнего контракта задач.

Приложение A. Каталог задач

См. `docs/bilingual_task_inventory.md` (41 семейство, en/ru, с `task_id` и путями).

Приложение B. Артефакты экспериментов

- Канонический раунд: `runs/canonical_round1/` (summary и логи по агентам и срезам).
- Ранние dev-матрицы и история исправлений постановок: `runs/public_agent_smoke/REPORT.md`, `runs/hidden_agent_smoke/REPORT.md` и история коммитов репозитория.